container are partitioned in a hierarchical sequence of minimum bounding boxes, one for each intermediate node, with the one at the root containing all the points, and the bounding box of each node fully containing all its children's bounding boxes. In chapter 10 we will give a brief description of how R-trees work.

- **SS-tree**: Similar to R-trees, but *Similarity Search Trees* use hyper-spheres as bounding regions. Hyper-spheres are constructed with a recursive structure: the leaves only contain points, while inner spheres' children are other hyper-spheres. Either way, a hyper-sphere can gather up to a certain number n of points (or spheres), within a certain distance from the sphere's center: when it gets more than n children or some of them are too far away with respect to the others, then the tree is rebalanced. We'll describe in details how this is done in the chapter 10, which is devoted to SS-trees.

And finally, let's define a generic interface, common to all concrete implementations.

Abstract Data Structure: NearestNeighborContainer	

API	`class NearestNeighborContainer {`
	`  size(),`
	`  isEmpty(),`
	`  insert(point),`
	`  remove(point),`
	`  search(point),`
	`  nearestNeighbor(point),`
	`  pointsInRegion(targetRegion)`
	`}`

Contract with client	The container allows inserting and removing points, and the following queries: Existence: check if a point is in the container.